

UBI Advanced Programme – Ethical Hacking Advanced Stage (Project 5)

Reconnaissance Engine

By: Akinkoye Oluwafemi

INTERN ID: UBI-2026-0588

Programming Language: Python 3.11

Date: 23rd July 2026

This report presents the implementation and testing of a scope-aware reconnaissance engine developed for the UBI Advanced Programme Ethical Hacking Advanced Stage assessment. The engine was designed to perform authorized reconnaissance within the provided laboratory environment while complying with the Rules of Engagement. It validates the assigned scope, communicates with the authorized services, preserves evidence, and retrieves the approved foothold required for the assessment. The following sections describe the development process, testing, and evidence generated during implementation.

1.1 Project Setup and Environment Preparation

The implementation of the reconnaissance engine started with preparing the development environment. A dedicated project folder named **ReconEngine** was created to organize all source code, project files, and generated outputs. A Python virtual environment (**venv**) was also created to isolate the project dependencies from other Python projects on the computer. Although the project mainly uses Python's standard library, creating a virtual environment provides a clean and controlled development environment and follows good software development practice.

The assessment package supplied for the assignment was downloaded and extracted from the compressed archive. The extracted package contained the local laboratory, runtime files, parser fixtures, rules of engagement, and other supporting documents required for the assessment. To keep the project organized, the assessment files were stored separately from the reconnaissance engine source code while still allowing the engine to access the required runtime files during execution.

Development was carried out using Visual Studio Code as the primary code editor. The reconnaissance engine was executed and tested through Windows PowerShell, where the command-line interface, unit tests, and runtime execution were verified throughout the implementation process.

Key Activities

- Created the **ReconEngine** project directory.
- Created and activated the Python virtual environment.
- Downloaded and extracted the assessment package.
- Organized the project files and assessment files separately.
- Prepared the development environment for implementation.

1.2 Understanding the Assessment Requirements

Before writing any code, the assessment documentation was carefully reviewed to understand the expected behaviour of the reconnaissance engine. As part of the preparation process, the SHA-256 hash of the supplied assessment package was verified to confirm that the downloaded files matched the expected integrity value before they were used. Particular attention was then given to the START-HERE guide, Rules of Engagement, and the laboratory instructions. These documents explained that the reconnaissance engine must only communicate with authorized loopback services and must never interact with systems outside the approved scope.

The assessment also introduced two important runtime files generated by the laboratory: `assignment.json` and `scope.csv`. The `assignment.json` file contains runtime information such as the profile, runtime identifier, request budget, authorized ports, and maximum request rate. The `scope.csv` file specifies which targets are within the approved scope (IN) and which targets are prohibited (OUT).

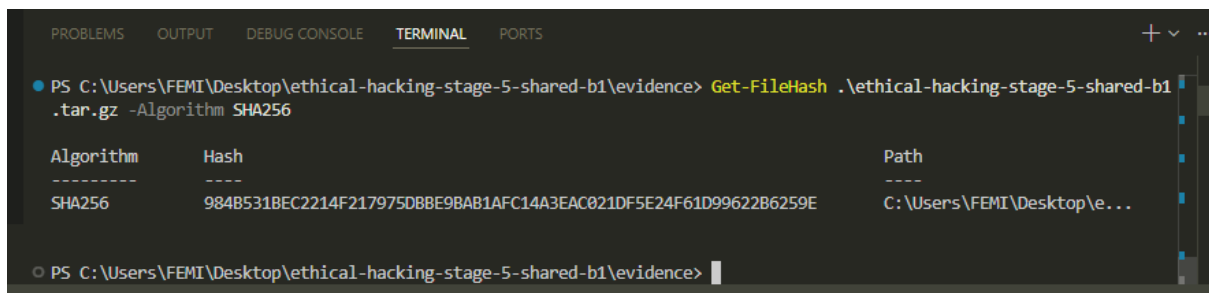
Intern: UBI-2026-0588 Track: ETHICAL_HACKING Base artifact: shared track release Private assignment
set: D5 Evidence marker: UBI-A5-86969972F9D6

Assigned evidence archive

- Filename: `ethical-hacking-stage-5-shared-b1.tar.gz`
- Bytes: 7694
- SHA-256: `984b531bec2214f217975dbbe9bab1afc14a3eac021df5e24f61d99622b6259e`
- Manifest signature: `6137fee074fa8d86a46154add763bd3f4adf9205ff479c30dd3e549f2ab77e6a`
- Download expiry: 2026-07-24T17:10:00.000Z

Figure 1.1: Review the assigned evidence

Understanding these files was important because they control how the reconnaissance engine operates and ensure that all network requests remain within the permitted assessment boundaries.



```
PS C:\Users\FEMI\Desktop\ethical-hacking-stage-5-shared-b1\evidence> Get-FileHash .\ethical-hacking-stage-5-shared-b1
.tar.gz -Algorithm SHA256

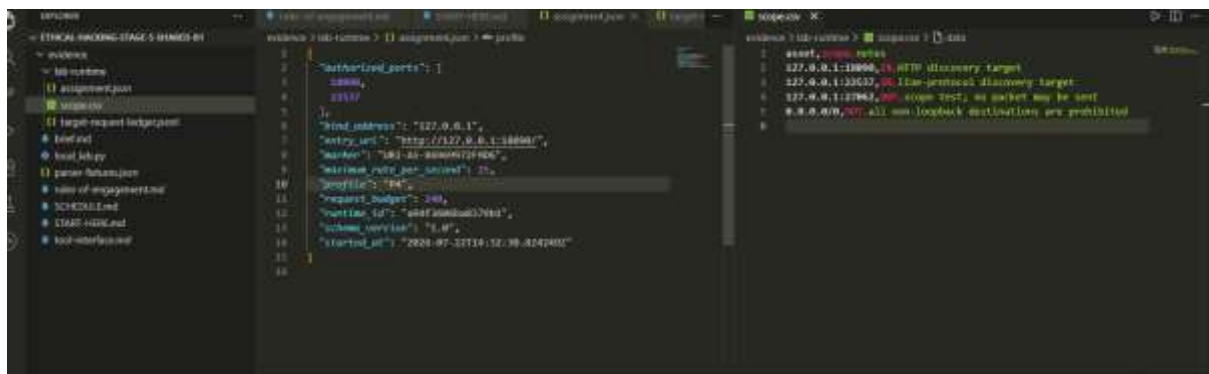
Algorithm      Hash                                                    Path
-----
SHA256         984B531BEC2214F217975DBBE9BAB1AFC14A3EAC021DF5E24F61D99622B6259E  C:\Users\FEMI\Desktop\e...
```

Figure 1.2: Verifying the Document hash

The supplied evidence package was opened, and the laboratory environment was initialized by executing the provided command:

```
python3 local_lab.py --marker UBI-A5-86969972F9D6 --output lab-runtime
```

Running this command generated the lab-runtime directory containing the assessment-specific runtime files, including assignment.json and scope.csv. These files provided the authorized runtime configuration, including the assigned ports (18090 and 22537), request budget, runtime identifier, and other information required for the reconnaissance engine.



Key Activities

- Reviewed the assessment documentation.
- Verified the SHA-256 hash of the supplied assessment package before use.
- Studied the Rules of Engagement.
- Examined the laboratory runtime files.
- Understood the contents of **assignment.json**.
- Understood the contents of **scope.csv**.

1.3 Creating the Project Structure

After understanding the project requirements, the directory structure for the reconnaissance engine was created. Instead of writing the entire application in a single Python file, the project was divided into multiple modules, with each module performing a specific responsibility. This

modular approach improves readability, simplifies maintenance, and allows new features to be added without affecting unrelated parts of the application.

The project package contains modules responsible for configuration loading, scope validation, network communication, data models, utility functions, and the main program logic. Separate folders were also created to store raw evidence, normalized output, automated test cases, and supporting project resources. This structure keeps the implementation organized and follows standard software engineering practices.

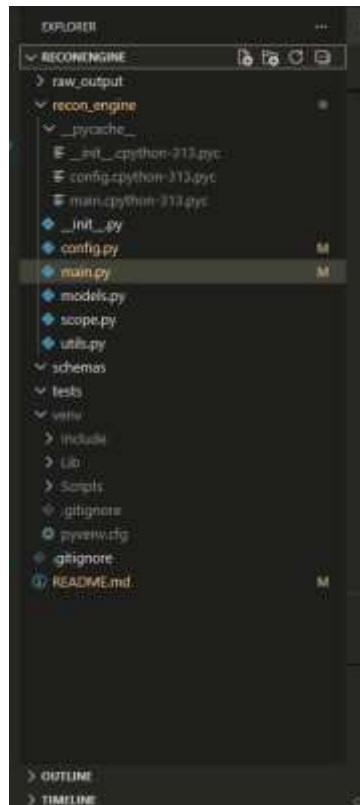


Figure 1.3: Project directory structure in Visual Studio Code.

Key Activities

- Created the ReconEngine project structure.
- Organized the source code into separate modules.
- Created folders for raw evidence and normalized output.
- Added folders for automated tests and project resources.
- Structured the project using a modular design.

1.4 Command-Line Interface Development

The first functional component implemented was the command-line interface (CLI). Python's **argparse** library was used to allow the reconnaissance engine to receive runtime parameters directly from the command line instead of modifying the source code before every execution.

The completed CLI accepts four required parameters: the target address, the scope file, the output directory, and the maximum request rate. Before execution begins, the supplied arguments are validated automatically to ensure that all required values have been provided. If a required argument is missing, the program displays an appropriate error message together with the built-in help screen.

The command-line interface also allows the same reconnaissance engine to operate with different assessment environments simply by supplying different runtime parameters.

Key Activities

- Implemented the command-line interface using **argparse**.
- Accepted the target, scope, output, and rate parameters.
- Validated required command-line arguments.
- Displayed the supplied runtime configuration.
- Improved flexibility by supporting configurable execution.

1.5 Reading Runtime Configuration

After processing the command-line arguments, the reconnaissance engine loads the runtime configuration generated by the assessment laboratory. A dedicated configuration module was developed to locate and read the **assignment.json** file automatically.

Instead of requiring the user to specify the location of the runtime configuration separately, the engine determines its location using the supplied **scope.csv** file. Once the configuration file is successfully loaded, important runtime information such as the assessment profile, runtime identifier, request budget, maximum request rate, and authorized ports is displayed to the user before any network activity begins.

The implementation also verifies that the configuration file exists before attempting to read it, preventing runtime errors caused by missing files.

```

41: def main():
42:     args = parse_arguments()
43:     from pathlib import Path
44:     scope_path = Path(args.scope)
45:     assignment_path = scope_path.with_name("assignment.json")
46:
47:     assignment = load_assignment(str(assignment_path))
48:     scope = load_scope(args.scope)
49:     if not is_target_allowed(args.target, scope):
50:         print("[ERROR] Target: {args.target} is outside the authorized scope.")
51:         return
52:
53:     print("=== Recon Engine ===")
54:     print(f"Target : {args.target}")
55:     print(f"Scope : {args.scope}")
56:     print(f"Output : {args.output}")
57:     print(f"Rate : {args.rate} req/sec")
58:
59:     print("=== Assignment Information ===")
60:     print(f"Profile : {assignment['profile']}")
61:     print(f"Runtime ID : {assignment['runtime_id']}")
62:     print(f"Request Budget : {assignment['request_budget']}")
63:     print(f"Runtime Rate : {assignment['runtime_rate_per_second']}")
64:     print(f"Authorized Ports : {assignment['authorized_ports']}")
65:
66: if __name__ == "__main__":
67:     main()

```

```

PS C:\Users\JPH\Desktop> python .\recon_engine.py --target 127.0.0.1:18000 --scope ".\artificial-hacking-stage-1-shared\1\recon\lab\recon-engine.json"
Output: run - rate: 25
=== Assignment Information ===
Profile : PR
Runtime ID : 52455C4B101046
Request Budget : 250
Runtime Rate : 25
Authorized Ports : [18000, 2255]
PS C:\Users\JPH\Desktop> python .\recon_engine.py --target 127.0.0.1:27002 --scope ".\artificial-hacking-stage-1-shared\1\recon\lab\recon-engine.json"
Output: run - rate: 25
[ERROR] Target: 127.0.0.1:27002 is outside the authorized scope.

```

Figure 1.4: Loading and displaying the runtime configuration from assignment.json.

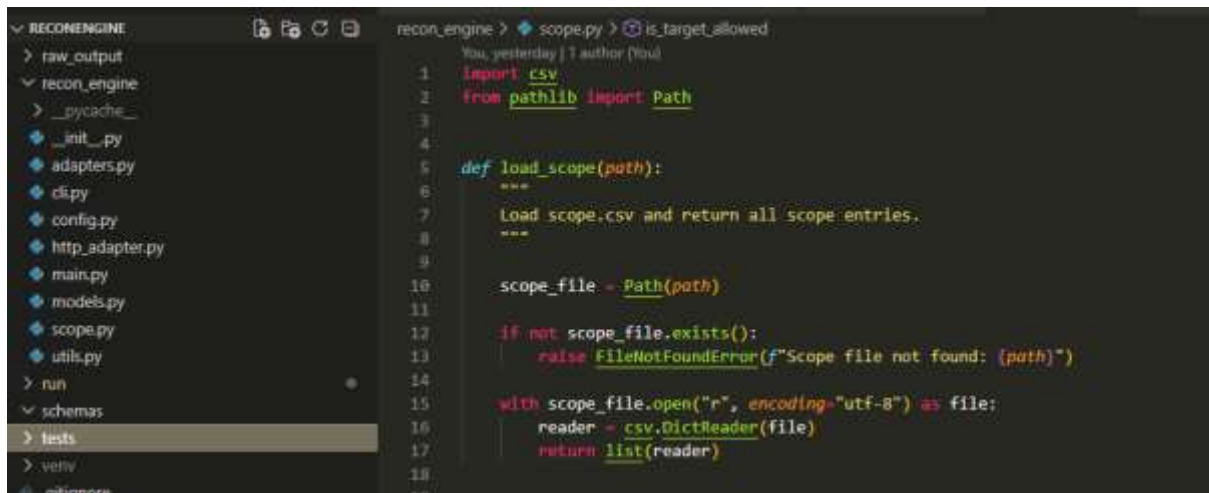
Key Activities

- Loaded the runtime configuration automatically.
- Verified that **assignment.json** exists.
- Parsed the JSON configuration.
- Displayed runtime information to the user.
- Automatically located the configuration file using **scope.csv**.

1.6 Scope File Processing

To ensure compliance with the assessment rules, the reconnaissance engine processes the supplied **scope.csv** file before any network communication begins. Python's **csv.DictReader** was used to read the CSV file and convert each record into a dictionary, making it easier to access fields such as the target, scope status, and additional notes.

The processed records are stored in memory as a collection of dictionaries and later used during scope validation. This allows the engine to quickly determine whether a supplied target is authorized before attempting to establish a network connection. Processing the scope file at the beginning of execution helps enforce the Rules of Engagement and prevents accidental communication with unauthorized destinations.



```
recon_engine > scope.py > is_target_allowed
You, yesterday | 1 author (You)
1 import csv
2 from pathlib import Path
3
4
5 def load_scope(path):
6     """
7     Load scope.csv and return all scope entries.
8     """
9
10    scope_file = Path(path)
11
12    if not scope_file.exists():
13        raise FileNotFoundError(f"Scope file not found: {path}")
14
15    with scope_file.open("r", encoding="utf-8") as file:
16        reader = csv.DictReader(file)
17        return list(reader)
18
```

Figure 1.5: Reading and processing the *scope.csv* file.

Key Activities

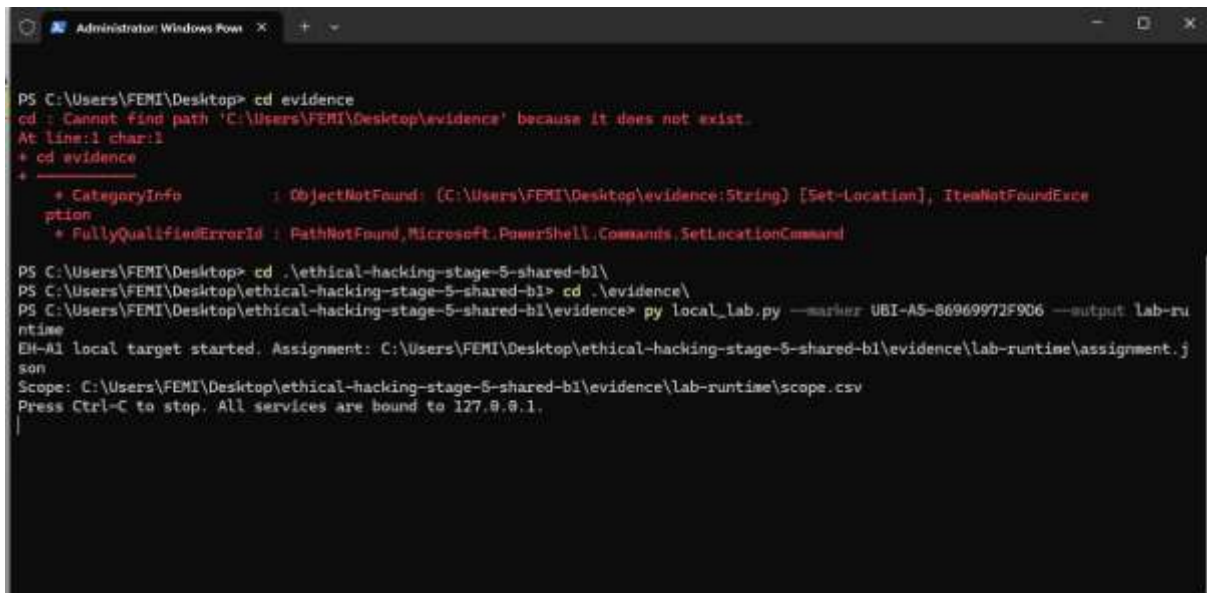
- Opened and processed **scope.csv**.
- Parsed CSV records using **csv.DictReader**.
- Converted records into dictionaries.
- Stored scope information in memory.
- Prepared the data for scope validation.

1.7 Scope Validation

After processing the scope file, a scope validation module was implemented to ensure that only authorized targets could be accessed. Before any network connection is established, the supplied target is compared with the approved entries contained in **scope.csv**. The engine also verifies that the requested port is one of the authorized ports specified in **assignment.json**.

If the target and port are within the approved scope, execution continues normally. However, if the destination is outside the approved scope, uses an unauthorized port, or is not found in the scope file, the program immediately stops before opening any socket or sending an HTTP request. This behaviour follows the Rules of Engagement and ensures that no unauthorized network activity is performed.

To verify this behaviour, automated unit tests were created for valid targets, invalid ports, non-loopback addresses, and out-of-scope destinations. All validation tests passed successfully before the discovery components were implemented.



```
Administrator: Windows PowerShell
PS C:\Users\FEMI\Desktop> cd evidence
cd : Cannot find path 'C:\Users\FEMI\Desktop\evidence' because it does not exist.
At line:1 char:1
+ cd evidence
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\FEMI\Desktop\evidence:String) [Set-Location], ItemNotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand

PS C:\Users\FEMI\Desktop> cd .\ethical-hacking-stage-5-shared-b1\
PS C:\Users\FEMI\Desktop\ethical-hacking-stage-5-shared-b1> cd .\evidence\
PS C:\Users\FEMI\Desktop\ethical-hacking-stage-5-shared-b1\evidence> py local_lab.py --marker UBI-A5-86969972F906 --output lab-runtime
EH-All local target started. Assignment: C:\Users\FEMI\Desktop\ethical-hacking-stage-5-shared-b1\evidence\lab-runtime\assignment.json
Scope: C:\Users\FEMI\Desktop\ethical-hacking-stage-5-shared-b1\evidence\lab-runtime\scope.csv
Press Ctrl-C to stop. All services are bound to 127.0.0.1.
```

Figure 1.6: The local target running on a terminal

Key Activities

- Compared user-supplied targets against **scope.csv**.
- Verified authorized ports.
- Rejected unauthorized destinations before network communication.
- Applied the default-deny principle.
- Tested scope validation using automated unit tests.

1.8 Establishing Communication with the Target

Once the supplied target successfully passed the scope validation checks, the reconnaissance engine established communication with the authorized service. A dedicated socket adapter was implemented to separate the networking logic from the rest of the application. This modular design makes the code easier to maintain and allows additional communication protocols to be integrated without changing the main application flow.

The engine parses the supplied target into its host and port components before creating a TCP socket connection. After successfully connecting to the service, the engine confirms that communication has been established before proceeding with further protocol interactions.

All socket operations are performed using appropriate timeout handling to prevent the application from waiting indefinitely for a response. After completing each operation, the connection is closed safely to release system resources.

A terminal window with a dark background and light green text. The prompt is '(venv) PS C:\Users\VEFI\Desktop\ReconEngine>'. The command entered is 'python recon_engine.py --target 127.0.0.1:22537 --scope ../ethical-hacking-stage-5-shared-b3/evidence/lab-runtime/scope.csv'. The output shows 'Output: run --rate 25', 'Rate : 25 req/sec.', and a section '=== Assignment Information ===' with details: Profile: P4, Runtime ID: a94f360b837961, Request Budget: 240, Maximum Rate: 25, and Authorized Ports: [18090, 22537]. It then shows 'Connecting to target...' and '[SUCCESS] Connected to target.'

```
(venv) PS C:\Users\VEFI\Desktop\ReconEngine> python recon_engine.py --target 127.0.0.1:22537 --scope ../ethical-hacking-stage-5-shared-b3/evidence/lab-runtime/scope.csv
Output: run --rate 25
Rate : 25 req/sec.

=== Assignment Information ===
Profile : P4
Runtime ID : a94f360b837961
Request Budget : 240
Maximum Rate : 25
Authorized Ports : [18090, 22537]

Connecting to target...
[SUCCESS] Connected to target.
```

Figure 1.7: Successful TCP connection with the authorized target.

Key Activities

- Parsed the target host and port.
- Established a TCP socket connection.
- Implemented timeout handling.
- Confirmed successful communication.
- Closed socket connections safely.

1.9 Output Generation and Evidence Recording

To support the evidence requirements of the assessment, the reconnaissance engine automatically creates the required output directory structure during execution. Separate directories are maintained for raw protocol responses and normalized observations, ensuring that original evidence remains unchanged while processed results are stored independently.

At the beginning of every execution, the engine generates a **run.json** file containing important execution information such as the execution time, runtime profile, supplied command-line arguments, and overall execution status. Runtime errors and unexpected events are recorded separately in **errors.jsonl** using the JSON Lines format, allowing new records to be added efficiently without modifying previous entries.

This structured output makes it easier to review execution results and provides clear evidence of the reconnaissance activities performed by the engine.

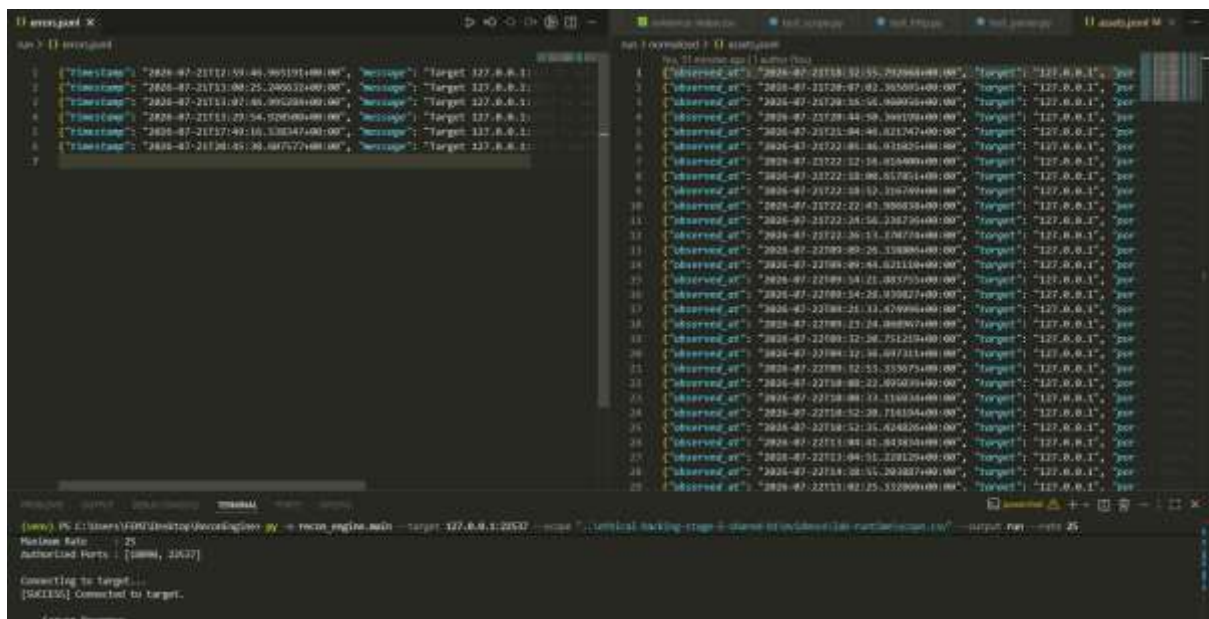


Figure 1.8: Automatically generated execution artifacts.

Key Activities

- Created the required output directories.
- Generated **run.json** automatically.
- Recorded runtime errors in **errors.jsonl**.
- Preserved execution evidence.
- Organized raw and normalized outputs separately.

1.10 Normalized Asset Recording

A standardized data model was implemented to ensure that every observation collected during reconnaissance is stored using a consistent structure. A Python **dataclass** named **AssetRecord** was created to represent each discovered asset regardless of the protocol used during discovery. Each normalized record contains the observation time, target address, port number, transport protocol, identified service, source tool, source file, confidence level, and supporting notes. After every successful observation, the engine automatically writes the record to **normalized/assets.jsonl** using the JSON Lines format.

Using a normalized structure makes it easier to process reconnaissance results, compare observations from different discovery adapters, and generate reports while preserving a consistent evidence format.

Key Activities

- Designed the **AssetRecord** data model.
- Standardized reconnaissance observations.

- Generated **assets.jsonl** automatically.
- Stored structured asset information.
- Preserved normalized evidence.

1.11 Version Control and Progress Tracking

Throughout the implementation process, Git was used to manage version control and record development progress. Major milestones were committed regularly, allowing changes to be tracked throughout the project. Using version control also made it possible to recover previous versions whenever necessary and provided a complete history of the engine's development.

As new features were completed and tested successfully, the repository was synchronized with a remote Git repository to maintain a secure backup of the project. Each commit documented a meaningful stage of implementation, including improvements to scope validation, protocol communication, automated evidence generation, and the command-line interface.

Maintaining version control throughout the project improved organization, supported reproducibility, and demonstrated the gradual development of the reconnaissance engine.

1.12 Multi-Protocol Discovery

After successfully establishing communication with the authorized services, the reconnaissance engine began collecting information from both services provided by the assessment laboratory. The engine supports communication with the custom line-protocol service and the HTTP service, allowing information gathered from one service to be used during communication with the other.

The discovery process starts by connecting to the line-protocol service. Once the connection is established, the engine receives the service banner, confirming that the service is available and ready to accept requests. The engine then sends the supported protocol commands to obtain additional information required during the reconnaissance process.

For the HTTP service, the engine sends requests to the authorized web service and records the responses. HTTP status codes, response headers and response bodies are preserved as raw evidence while useful observations are written into the normalized output files. Keeping both raw and normalized evidence makes it possible to verify every observation collected during execution.

The modular design allows both discovery adapters to work independently while producing results using the same normalized format.

1.13 Route Discovery and Virtual Host Identification

After receiving the initial banner from the line-protocol service, the reconnaissance engine continued interacting with the protocol by sending supported commands. One of the most important commands implemented was the **ROUTE** command.

The server responded by returning the assigned virtual host together with the corresponding route proof. These values were extracted automatically and stored temporarily by the engine because they were required during the next stage of the reconnaissance process.

Using the discovered virtual host, the engine sent HTTP requests with the correct **Host** header instead of accessing only the default web page. This allowed the engine to reach the authorized virtual host allocated to the assessment profile. The successful retrieval of the virtual host confirmed that the engine was correctly following the intended discovery workflow rather than relying on hard-coded values.

```
[SUCCESS] Connected to target.

=== Server Response ===
RLY/2 READY profile=dynamic

=== ROUTE Response ===
route=archive-7bc8470.switchyard.local; proof=cad557058869576c

=== Virtual Host Test ===
200
<title>Relay Operations</title><h1>Relay Operations</h1>

Route : archive-7bc8470.switchyard.local
Proof : cad557058869576c
```

Figure 1.10: Retrieving the assigned virtual host and route proof.

Key Activities

- Received the protocol banner.
- Implemented the **ROUTE** command.
- Extracted the virtual host automatically.
- Retrieved the route proof.
- Used the virtual host during HTTP communication.

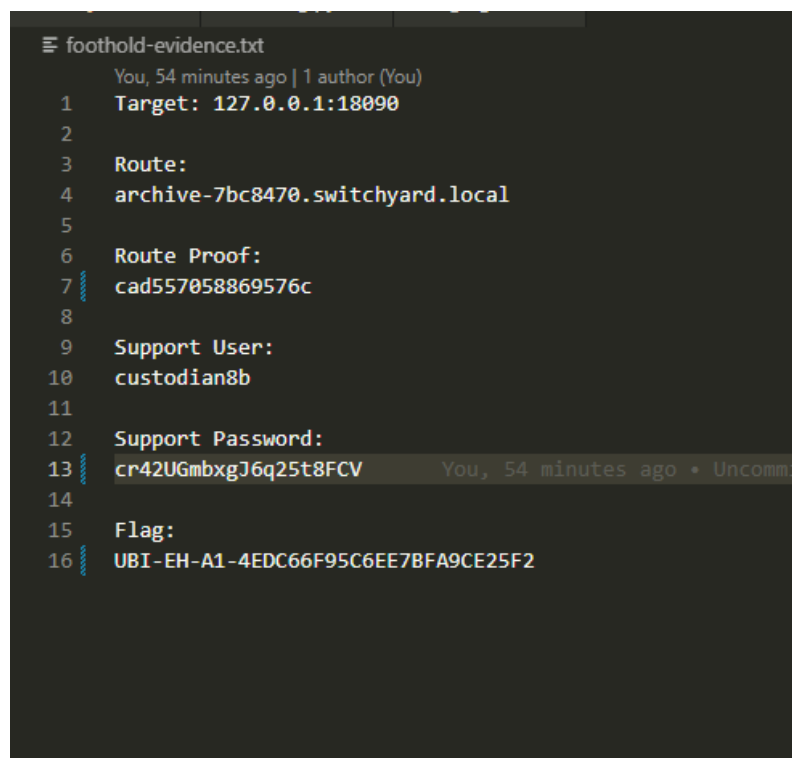
1.14 Foothold Discovery Process

After identifying the correct virtual host, the reconnaissance engine continued the authorized discovery process by requesting the **/ops-diagnostics** endpoint. This endpoint returned additional diagnostic information, including the support username and password required for authentication. It also confirmed that the route proof originated from the signal service.

The retrieved username and password were automatically converted into a Basic Authentication header, while the previously discovered route proof was included in the required **X-Route-Key** HTTP header. These values were then used to request the protected **/user.txt** resource.

Once the authentication credentials and route proof were successfully verified by the server, the protected resource returned the assigned foothold flag. The engine recorded this result as evidence and stored the required outputs for later verification.

The engine immediately stopped after obtaining the foothold flag. No additional exploitation, privilege escalation, or unauthorized requests were performed. This behaviour follows the Rules of Engagement supplied with the assessment.



```
foothold-evidence.txt
You, 54 minutes ago | 1 author (You)
1 Target: 127.0.0.1:18090
2
3 Route:
4 archive-7bc8470.switchyard.local
5
6 Route Proof:
7 cad557058869576c
8
9 Support User:
10 custodian8b
11
12 Support Password:
13 cr42UGmbxgJ6q25t8FCV You, 54 minutes ago • Uncommi
14
15 Flag:
16 UBI-EH-A1-4EDC66F95C6EE7BFA9CE25F2
```

Figure 1.11: Successful retrieval of the authorized foothold flag.

Key Activities

- Requested the **/ops-diagnostics** endpoint.
- Retrieved the support credentials.
- Generated the Basic Authentication header.
- Included the **X-Route-Key** header.

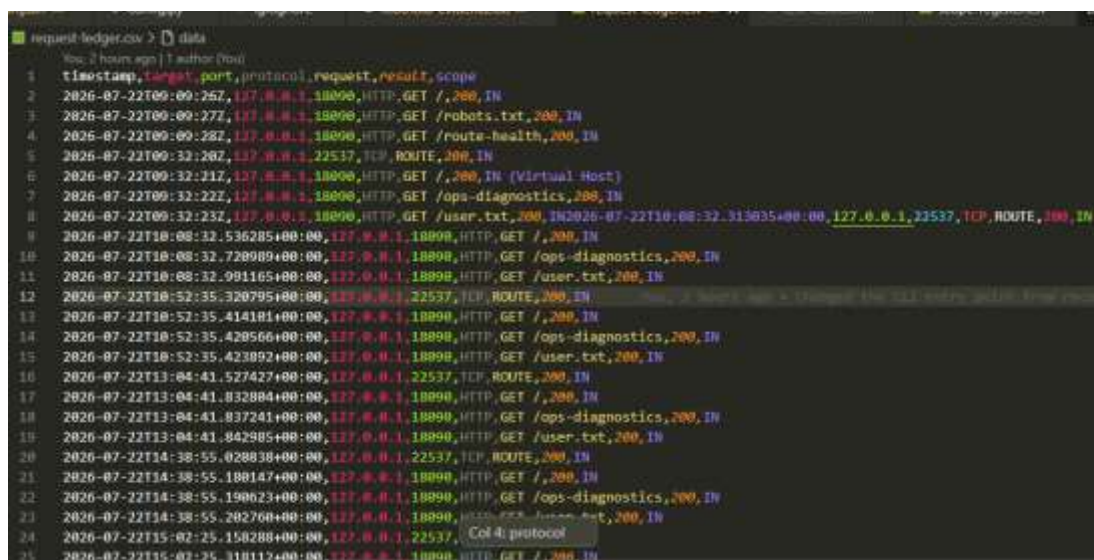
1.15 Automatic Evidence Generation

To improve traceability and support the assessment requirements, the reconnaissance engine automatically generates the required evidence during execution instead of relying on manual record keeping.

Every authorized request made by the engine is recorded automatically in **request-ledger.csv**. Each record contains the request time, destination, protocol, request performed, response result, and scope decision. This provides a complete audit trail of the engine's activities and demonstrates that every request remained within the approved assessment scope.

The engine also preserves raw protocol responses inside the **run/raw** directory while normalized observations are written to **run/normalized/assets.jsonl**. Additional execution information is stored in **run.json**, runtime errors are recorded in **errors.jsonl**, and the recovered foothold flag is preserved separately in **foothold-evidence.txt**.

Maintaining both raw and normalized evidence improves transparency because every normalized observation can be traced back to its original response.



```
request-ledger.csv > data
1 timestamp,target,port,protocol,request,result,scope
2 2026-07-22T00:00:26Z,127.0.0.1,18090,HTTP,GET /,200,IN
3 2026-07-22T00:00:27Z,127.0.0.1,18090,HTTP,GET /robots.txt,200,IN
4 2026-07-22T00:00:28Z,127.0.0.1,18090,HTTP,GET /route-health,200,IN
5 2026-07-22T00:32:20Z,127.0.0.1,22537,TCP,ROUTE,200,IN
6 2026-07-22T00:32:21Z,127.0.0.1,18090,HTTP,GET /,200,IN (Virtual Host)
7 2026-07-22T00:32:22Z,127.0.0.1,18090,HTTP,GET /ops-diagnostics,200,IN
8 2026-07-22T00:32:23Z,127.0.0.1,18090,HTTP,GET /user.txt,200,IN
9 2026-07-22T10:08:32.536285+00:00,127.0.0.1,18090,HTTP,GET /,200,IN
10 2026-07-22T10:08:32.728089+00:00,127.0.0.1,18090,HTTP,GET /ops-diagnostics,200,IN
11 2026-07-22T10:08:32.981165+00:00,127.0.0.1,18090,HTTP,GET /user.txt,200,IN
12 2026-07-22T10:52:35.320795+00:00,127.0.0.1,22537,TCP,ROUTE,200,IN
13 2026-07-22T10:52:35.414181+00:00,127.0.0.1,18090,HTTP,GET /,200,IN
14 2026-07-22T10:52:35.428566+00:00,127.0.0.1,18090,HTTP,GET /ops-diagnostics,200,IN
15 2026-07-22T10:52:35.423892+00:00,127.0.0.1,18090,HTTP,GET /user.txt,200,IN
16 2026-07-22T13:04:41.527422+00:00,127.0.0.1,22537,TCP,ROUTE,200,IN
17 2026-07-22T13:04:41.832884+00:00,127.0.0.1,18090,HTTP,GET /,200,IN
18 2026-07-22T13:04:41.837241+00:00,127.0.0.1,18090,HTTP,GET /ops-diagnostics,200,IN
19 2026-07-22T13:04:41.842985+00:00,127.0.0.1,18090,HTTP,GET /user.txt,200,IN
20 2026-07-22T14:38:55.028838+00:00,127.0.0.1,22537,TCP,ROUTE,200,IN
21 2026-07-22T14:38:55.188147+00:00,127.0.0.1,18090,HTTP,GET /,200,IN
22 2026-07-22T14:38:55.198023+00:00,127.0.0.1,18090,HTTP,GET /ops-diagnostics,200,IN
23 2026-07-22T14:38:55.282768+00:00,127.0.0.1,18090,HTTP,GET /user.txt,200,IN
24 2026-07-22T15:02:25.158288+00:00,127.0.0.1,22537,,Col 4: protocol
25 2026-07-22T15:02:25.318112+00:00,127.0.0.1,18090,HTTP,GET /,200,IN
```

Figure 1.12: Automatically generated evidence produced by the reconnaissance engine.

TESTING AND EVALUATION

After completing the implementation of the reconnaissance engine, a series of tests were carried out to verify that every component functioned correctly and complied with the assessment requirements. The testing process focused on validating scope enforcement, command-line argument handling, protocol communication, normalized output generation, evidence preservation, and successful retrieval of the authorized foothold. Both automated unit tests and practical execution tests were performed throughout the development process.

```
PS C:\Users\FEMI\Desktop\ReconEngine> py -m unittest discover -s tests -v
test_empty_robots (test_http.TestHTTP.test_empty_robots) ... ok
test_extract_multiple_paths (test_http.TestHTTP.test_extract_multiple_paths) ... ok
test_extract_single_path (test_http.TestHTTP.test_extract_single_path) ... ok
test_parse_route (test_parser.TestParser.test_parse_route) ... ok
test_parse_target (test_parser.TestParser.test_parse_target) ... ok
test_allowed_http_port (test_scope.TestScope.test_allowed_http_port) ... ok
test_allowed_signal_port (test_scope.TestScope.test_allowed_signal_port) ... ok
test_non_loopback (test_scope.TestScope.test_non_loopback) ... ok
test_out_of_scope_port (test_scope.TestScope.test_out_of_scope_port) ... ok

-----
Ran 9 tests in 0.005s

OK
```

2.1 Scope Validation Testing

The first stage of testing focused on verifying that the engine correctly enforced the Rules of Engagement before any network communication occurred. Automated unit tests were written to confirm that only authorized loopback targets and approved ports were accepted.

The tests verified that requests directed to non-loopback addresses, unauthorized ports, and destinations outside the approved scope were rejected before any socket connection was established. This behaviour ensured that the engine complied with the assessment requirements and prevented accidental communication with unauthorized systems.

The completed scope validation tests all passed successfully.

Test Cases

- Authorized HTTP port accepted.
- Authorized line-protocol port accepted.
- Unauthorized port rejected.
- Non-loopback address rejected.

2.2 Parser Testing

Parser tests were implemented to verify that the reconnaissance engine correctly extracted useful information from server responses.

The HTTP parser was tested using multiple **robots.txt** examples to confirm that hidden paths could be extracted correctly. Additional parser tests verified that the engine correctly extracted routing information returned by the line-protocol service.

These tests ensured that protocol responses were interpreted correctly before they were used during later stages of the reconnaissance process.

Test Cases

- Single robots.txt entry.
- Multiple robots.txt entries.
- Route extraction.
- Target parsing.

2.3 Automated Unit Testing

After completing the individual modules, the entire automated test suite was executed using Python's **unittest** framework.

The test suite included scope validation tests, HTTP parser tests, and protocol parser tests. During the final execution, all nine published tests completed successfully without any failures.

The successful execution of the test suite confirmed that the core functionality of the reconnaissance engine operated as expected before practical testing was performed.

Test Summary

Test Category	Result
HTTP Parser	Passed
Route Parser	Passed
Scope Validation	Passed
Total Tests	9 Passed

2.4 Command-Line Interface Testing

The command-line interface was tested to verify that all required runtime arguments were accepted correctly.

The **--help** option displayed the available command-line parameters, while valid execution successfully accepted the target, scope file, output directory, and request rate supplied by the user.

Additional tests confirmed that the application reported appropriate errors whenever required arguments were omitted.

2.5 Functional Testing

Functional testing was carried out using the assessment laboratory provided for the project.

The engine successfully connected to the authorized line-protocol service, retrieved the service banner, obtained the assigned route and virtual host, communicated with the HTTP service, collected the diagnostic information, and finally retrieved the authorized foothold flag.

Throughout the execution, the engine automatically generated the required evidence files, including the normalized asset records, execution logs, request ledger, and foothold evidence.

The successful completion of these activities demonstrated that the reconnaissance engine achieved the required functionality while remaining within the approved assessment scope.

```
PS C:\Users\FEMI> cd Desktop
PS C:\Users\FEMI\Desktop> cd .\ReconEngine\
PS C:\Users\FEMI\Desktop\ReconEngine> py -w recon_engine.cli --target 127.0.0.1:18090 --scope '..\ethical-hacking-stage-5-shared-b1\evidence\lab-runtime\scope.csv' --output
un --rate 25
=== Recon Engine ===
Target : 127.0.0.1:18090
Scope : ..\ethical-hacking-stage-5-shared-b1\evidence\lab-runtime\scope.csv
Output : run
Rate : 25 req/sec

=== Assignment Information ===
Profile : RM
Runtime ID : a84f3608ba8379b2
Request Budget : 240
Maximum Rate : 25
Authorized Ports : [18090, 22637]

Performing HTTP discovery...

Path : /
Status : 200
Server : TransitGateway/2.0
Body : Transit edge online. Route is not provisioned.

Path : /robots.txt
Status : 200
Server : TransitGateway/2.0
Body : User-agent: *
Disallow: /route-health

Following discovered path: /route-health
Status : 200
Body : Transit edge online. Route is not provisioned.
```

```

=== Recon Engine ===
Target : 127.0.0.1:22537
Scope : ..\ethical-hacking-stage-5-shared-b1\evidence\lab-runtime\scope.csv
Output : run
Rate : 25 req/sec

=== Assignment Information ===
Profile : 94
Runtime ID : a84f3d4dhw8279b1
Request Budget : 248
Maximum Rate : 25
Authorized Ports : [18998, 22537]

Connecting to target...
[SUCCESS] Connected to target.

=== Server Response ===
RLY/2 READY profile=dynadn

=== ROUTE Response ===
route=archive-7bc8b7d.switchyard.local; proof=cad567b65d69576c

=== Virtual Host Test ===
200
<title>Relay Operations</title><h1>Relay Operations</h1>
Route : archive-7bc8b7d.switchyard.local
Proof : cad567b65d69576c

=== Diagnostics ===
200
{"note": "route proof is emitted by the signal service", "signal_service": 22537, "status": "degraded", "support_password": "cr42UGetopMq25t8fCV", "support_user": "custodian@e:"}

=== Flag ===
200
UBI-EH-A1-4EDC66F95C6EE78FA9CE29F2

PS C:\Users\FERN\Desktop\ReconEngine>

```

2.6 Output Verification

The generated evidence files were examined after each execution to verify that the required outputs had been created successfully.

The verification confirmed the successful generation of:

- run.json
- errors.jsonl
- normalized/assets.jsonl
- request-ledger.csv
- foothold-evidence.txt
- raw protocol responses

These files provided evidence that the engine completed the authorized reconnaissance process and preserved both raw and normalized observations for future analysis.

2.8 Summary

The testing activities confirmed that the reconnaissance engine satisfied the functional requirements of the assessment. Scope enforcement prevented unauthorized communication, automated tests verified the correctness of individual components, and practical execution demonstrated successful interaction with the authorized services. The engine successfully generated the required evidence, preserved normalized observations, and retrieved the authorized foothold while remaining within the approved Rules of Engagement.